Jobnity Codebase Method Reference

Detailed Process & Execution Logic for Every Backend & Service Function

FastAPI Endpoints | NLP & Parsers | Skill Gap Logic | Database ORM

1. REST API Route Handlers (backend/app/api/routes/)

POST /api/resumes/upload → upload_resume()

File: backend/app/api/routes/resumes.py

Receives uploaded resume file (PDF/DOCX), parses text, extracts candidate profile via NLP, and saves records to database.

1. Calls `validate_uploaded_file(file)` to check extension (.pdf/.docx), MIME type, and size (<10MB).
2. Ensures `settings.UPLOAD_DIR` exists safely (fallback to `/tmp/jobnity_uploads` on Vercel).
3. Writes file stream to storage disk under a unique UUID filename (e.g. `a1b2c3d4.pdf`).
4. Invokes `ResumeParserService.parse_file()` to extract raw text content.
5. Generates unique IDs: `resume_id = R_xxxx` and `profile_id = prof_xxxx`.
6. Creates and adds `Resume` model record to SQLite database.
7. Invokes `NLPExtractorService.extract_profile(raw_text, db)` to extract name, email, phone, education, experience, skills, projects, and certifications.
8. Creates and commits `CandidateProfile` record to database.
9. Returns `200 OK` response with parsed profile payload.

GET /api/resumes/{resume_id}/profile → get_candidate_profile()

File: backend/app/api/routes/resumes.py

Fetches parsed candidate profile by resume ID.

1. Queries `CandidateProfile` table filtering by `resume_id == resume_id`.
2. If record not found, raises `HTTP 404 Not Found` error.
3. Returns candidate profile object containing name, contact details, and skill array.

PUT /api/resumes/{resume_id}/profile → update_candidate_profile()

File: backend/app/api/routes/resumes.py

Updates candidate profile fields based on user edits during profile review.

1. Fetches existing candidate profile from database by `resume_id`.
2. Updates non-null fields provided in `CandidateProfileUpdate` request body.
3. Overwrites candidate skill array — **user-confirmed skills become the single source of truth**.
4. Commits changes to database and refreshes session.
5. Returns updated candidate profile.

`POST /api/analysis/skill-gap → analyze_skill_gap()`

File: backend/app/api/routes/analysis.py

Executes weighted skill gap analysis between candidate profile and target job role.

1. Validates that `resume_id` and `role_id` (or target_role string) are provided.
2. Calls `SkillGapService.calculate_skill_gap(db, resume_id, role_id)`.
3. Returns structured analysis response containing readiness score, gap score, matched skills, and missing skills.

`GET /api/analysis/{analysis_id}/recommendations → get_analysis_recommendations()`

File: backend/app/api/routes/analysis.py

Fetches personalized learning resources for skills identified as missing in analysis.

1. Calls `RoadmapService.get_recommendations_for_analysis(db, analysis_id)`.
2. Queries `LearningResource` database table for each missing skill.
3. Returns step-by-step roadmap milestones with direct links to courses, documentation, and videos.

2. Document Text Parsers (backend/app/services/resume_parser.py)

`extract_text_from_pdf(file_path: str) → str`

Class: ResumeParserService

Extracts raw text from PDF files page-by-page.

1. Initializes `pypdf.PdfReader(file_path)` object.
2. Iterates through all document pages calling `page.extract_text()`.
3. Appends non-empty text chunks to a list and joins them with newlines.
4. Passes concatenated string through `clean_text()` utility.
5. Throws `HTTP 422 Unprocessable Entity` if PDF is scanned, empty, or unreadable.

`extract_text_from_docx(file_path: str) → str`

Class: ResumeParserService

Extracts raw text from Microsoft Word DOCX files.

1. Initializes `docx.Document(file_path)` instance.
2. Extracts paragraph text array from `doc.paragraphs`.
3. Iterates through all table cells (`doc.tables`) to extract text inside tables/lists.
4. Joins text paragraphs and cleans whitespace via `clean_text()`.
5. Returns cleaned text string.

3. NLP Extraction & Taxonomy Engine (backend/app/services/nlp_extractor.py)

`extract_skills_from_text(text: str, db: Session) → List[str]`

Class: NLPExtractorService

Scans raw resume text against database canonical skills and aliases.

1. Queries `Skill` table for all canonical skill names (1,156 skills).
2. Queries `SkillAlias` table for synonym mappings (1,321 aliases).
3. Builds a unified lookup dictionary `canonical_map[term.lower()] = canonical_name`.
4. Sorts search terms by string length descending to prevent substring false positives (e.g. `C` matching `C++`).
5. Applies regex word boundary pattern matching (`\bterm\b`) against lowercased resume text.
6. Returns sorted array of unique detected canonical skills.

`extract_name(header_text: str, full_text: str) → str`

Class: NLPExtractorService

Extracts candidate's full name from resume header.

1. Splits top lines of resume header into individual text lines.
2. Ignores lines containing emails (`@`), phone numbers, or words like "Resume" / "CV".
3. Picks the first line with 1-4 capitalized words as the candidate name.
4. Falls back to `"Candidate"` if no valid line is found.

4. Skill Gap Engine & Guardrails (backend/app/services/skill_gap.py)

`is_distinct_guardrail_violation(cand_skill: str, req_skill: str) → bool`

Class: SkillGapService

Enforces non-equivalent skill boundaries to prevent false-positive matching.

1. Normalizes candidate skill and required skill to lowercase strings.
2. Checks against guardrail sets: `{"java", "javascript"}`, `{"docker", "kubernetes"}`, `{"aws", "azure"}`, `{"python", "pytorch"}`, `{"c", "c++"}`, `{"react", "react native"}`, `{"sql", "nosql"}`.
3. Returns `True` if candidate skill and required skill belong to the same guardrail set but are not identical.

`calculate_skill_gap(db: Session, resume_id: str, role_id: str) → Dict`

Class: SkillGapService

Computes weighted readiness score, skill gap score, matched skills, and missing skills.

1. Fetches candidate profile by `resume_id` and job role by `role_id`.
2. Normalizes candidate skills array using `SkillNormalizerService`.
3. Fetches required and preferred `RoleSkill` requirements for target role.
4. Iterates role skills, checking matches via `match_skill()` while respecting distinct guardrails.
5. Calculates skill weight: `weight = priority * (1.5 if required else 1.0)`.
6. Calculates `Readiness Score = (Earned Weights / Total Weights) * 100`.
7. Calculates `Skill Gap Score = 100 - Readiness Score`.
8. Saves `SkillGapAnalysis` record to database and returns results payload.